

AnalysisHub — UI Customization Manual

How to Edit the Popup Window Appearance in main.py

This manual explains every section of the Repository Manager popup window, where it lives in main.py, and exactly what to change to customize it.

All line references are approximate — search for the quoted text in Notepad++ using Ctrl+F to find it instantly.

PART 1 — Global Settings (affect the ENTIRE window)

These are defined near the top of main.py, above all the window classes. Change these first — they cascade through every element automatically.

1A. Colors

Search for: `_CLR_ANSYS_BLUE`

python

```
_CLR_ANSYS_BLUE   = Drawing.Color.FromArgb(0, 120, 212)    # main blue – buttons, tit
_CLR_READY        = Drawing.Color.FromArgb(16, 124, 16)    # green – "Ready" status
_CLR_MISSING      = Drawing.Color.FromArgb(209, 52, 56)    # red – "MISSING" status
_CLR_ROW_ALT      = Drawing.Color.FromArgb(245, 247, 250)  # light blue-grey – altern
_CLR_SECTION_HDR  = Drawing.Color.FromArgb(220, 230, 245)  # section separator color
```

How colors work: `FromArgb(R, G, B)` — three numbers from 0-255.

- `FromArgb(0, 120, 212)` = ANSYS blue
- `FromArgb(255, 0, 0)` = pure red
- `FromArgb(0, 0, 0)` = black
- `FromArgb(255, 255, 255)` = white

Example — change the blue to a darker navy:

python

```
_CLR_ANSYS_BLUE = Drawing.Color.FromArgb(0, 70, 140)
```

1B. Fonts

Search for: `_FONT_NORMAL`

python

```
_FONT_NORMAL = Drawing.Font("Segoe UI", 9.5)           # most text in the window
_FONT_BOLD   = Drawing.Font("Segoe UI", 9.5, Drawing.FontStyle.Bold)  # bold text
_FONT_TITLE   = Drawing.Font("Segoe UI", 13, Drawing.FontStyle.Bold)  # "Analysis
_FONT_SMALL   = Drawing.Font("Segoe UI", 8.5)           # small labels, subtitle t
```

How fonts work: `Drawing.Font("Font Name", size, style)`

- Font name: any Windows font — "Segoe UI", "Arial", "Calibri", "Tahoma"
- Size: points (9.5 is standard, 13 is large heading)
- Style options: `Drawing.FontStyle.Bold`, `Drawing.FontStyle.Italic`, `Drawing.FontStyle.Regular` (or leave it out for Regular)

Example — make all normal text slightly larger:

python

```
_FONT_NORMAL = Drawing.Font("Segoe UI", 11)
_FONT_BOLD   = Drawing.Font("Segoe UI", 11, Drawing.FontStyle.Bold)
```

PART 2 — The Main Window (RepositoryForm)

This is the outer container — the window frame itself. Search for: `class RepositoryForm`

2A. Window Size

Search for: `self.Width = 1360`

python

```
self.Width = 1360    # starting width in pixels
self.Height = 900    # starting height in pixels
```

Example — make it open smaller:

python

```
self.Width = 1100
self.Height = 750
```

2B. Minimum Window Size (resize limit)

Search for: `self.MinimumSize`

```
python

self.MinimumSize = Drawing.Size(900, 600)
```

Users cannot drag the window smaller than this. Change both numbers to set a different minimum width and height.

2C. Window Title Bar Text

Search for: `self.Text = "Analysis Repository Manager"`

```
python

self.Text = "Analysis Repository Manager"
```

Change the text in quotes to whatever you want shown in the title bar.

2D. Window Background Color

Search for: `self.BackColor = Drawing.Color.White` (inside RepositoryForm)

```
python

self.BackColor = Drawing.Color.White
```

Change `Drawing.Color.White` to any color. Options:

- `Drawing.Color.White`
 - `Drawing.Color.WhiteSmoke`
 - `Drawing.Color.FromArgb(245, 248, 252)` — very light blue-grey
-

PART 3 — The Header (Title and Subtitle)

Search for:

python

```
# Title - "Analysis Repository"
lbl_h.Text      = "Analysis Repository"
lbl_h.Font      = _FONT_TITLE           # controls size/weight - edit _FONT_TITLE ab
lbl_h.ForeColor = _CLR_ANSYS_BLUE       # controls color - edit _CLR_ANSYS_BLUE abov
lbl_h.Location  = Drawing.Point(16, 10) # (pixels from left, pixels from top)

# Subtitle - smaller grey text below the title
lbl_sub.Text     = "Centralized file management for your ANSYS project"
lbl_sub.Font     = _FONT_SMALL
lbl_sub.ForeColor = Drawing.Color.Gray
lbl_sub.Location = Drawing.Point(18, 38)
```

What to change:

- `lbl_h.Text` — the main title text
 - `lbl_sub.Text` — the subtitle/description text
 - `lbl_sub.ForeColor` — subtitle color (currently gray)
 - `lbl_h.Location` / `lbl_sub.Location` — position as `Drawing.Point(X, Y)` where X = pixels from left edge, Y = pixels from top edge
-

PART 4 — The Toolbar Buttons

Search for:

Each button is created with `_make_btn(...)`. Here is the full set:

python

```

toolbar_y = 62    # vertical position of ALL buttons (pixels from top)
toolbar_h = 42    # height of ALL buttons

self._btn_add      = _make_btn("+ Add File(s)...", x=16, y=toolbar_y, w=160, h=toolbar_h)
self._btn_refresh  = _make_btn("🔄 Refresh", x=186, y=toolbar_y, w=110, h=toolbar_h)
self._btn_open     = _make_btn("▶ Open", x=306, y=toolbar_y, w=100, h=toolbar_h)
self._btn_remove   = _make_btn("✕ Remove", x=416, y=toolbar_y, w=110, h=toolbar_h)
self._btn_notes    = _make_btn("📝 Notes", x=536, y=toolbar_y, w=100, h=toolbar_h)
self._btn_health   = _make_btn("♥ Health Check", x=646, y=toolbar_y, w=140, h=toolbar_h)

```

_make_btn parameters explained:

Parameter	What it controls	Example
text	Button label	"My Button"
x	Distance from left edge	16
y	Distance from top edge	62
w	Button width in pixels	160
h	Button height in pixels	42
primary=True	Blue filled style (False = white outline style)	True / False

Important: The x position of each button must account for the width of the button to its left. If you make button 1 wider, shift all buttons after it to the right by the same amount.

Example — make all buttons taller:

```

python

toolbar_h = 50

```

Example — rename the Health Check button:

```

python

self._btn_health = _make_btn("🔍 Validate Files", 646, toolbar_y, 140, toolbar_h)

```

4A. How `_make_btn` Works (the button factory)

Search for: `def _make_btn`

python

```
def _make_btn(text, x, y, w=160, h=36, primary=False, handler=None):
    btn.FlatStyle = WinForms.FlatStyle.Flat      # flat/modern look
    if primary:
        btn.BackColor = _CLR_ANSYS_BLUE          # blue fill for primary buttons
        btn.ForeColor = Drawing.Color.White      # white text on blue
    else:
        btn.BackColor = Drawing.Color.White      # white fill for secondary buttons
        btn.ForeColor = Drawing.Color.FromArgb(33, 37, 41) # dark text
        btn.FlatAppearance.BorderColor = Drawing.Color.FromArgb(180, 180, 180) # gr
```

To change the color of ALL primary buttons at once, change `_CLR_ANSYS_BLUE`. To change the border color of ALL secondary buttons, change `FromArgb(180, 180, 180)`.

PART 5 — The Project Information Panel

Search for: `class ProjectInfoPanel`

This is the collapsible panel below the toolbar showing Title, Customer, Analyst, Status, and Revision fields.

5A. Panel Background Color

Search for: `self.BackColor = Drawing.Color.FromArgb(245, 248, 252)` (inside `ProjectInfoPanel.init`)

python

```
self.BackColor = Drawing.Color.FromArgb(245, 248, 252) # very light blue-grey
```

5B. Panel Header Text

Search for: `lbl.Text = "Project Information"`

python

```
lbl.Text = "Project Information"
lbl.Font = _FONT_BOLD
```

5C. Collapse/Expand Button

Search for: `btn_toggle.Text = u"\u25B2 Collapse"`

```
python

btn_toggle.Text = u"\u25B2 Collapse" # ▲ Collapse (when expanded)
# changes to:
self._btn_toggle.Text = u"\u25BC Expand" # ▼ Expand (when collapsed)
```

To change the arrow symbols, replace `\u25B2` and `\u25BC` with any Unicode character. For example `\u2212` is a minus sign, `\u002B` is a plus sign.

5D. Expanded and Collapsed Heights

Search for: `self.Size = Drawing.Size(1300, 106)` (inside `_toggle`)

```
python

# When expanded:
self.Size = Drawing.Size(1300, 106) # width=1300, height=106 pixels

# When collapsed:
self.Size = Drawing.Size(1300, 36) # width=1300, height=36 pixels (header bar only)
```

If you add more fields to the panel, increase 106 to make room.

5E. Input Field Sizes

Each text field is defined like this pattern:

```
python

tb.Location = Drawing.Point(x0 + col * 200 + 72, 4) # position
tb.Size     = Drawing.Size(118, 24)                # width=118, height=24
```

The fields are spaced 200 pixels apart (`col * 200`). To make fields wider, increase 118. To space them further apart, increase 200 (but also increase the overall panel width in the Size lines above).

5F. Status Dropdown Options

Search for: `for s in ["Active", "In Review", "Complete"]`

python

```
for s in ["Active", "In Review", "Complete", "On Hold", "Archived"]:  
    self._cb_status.Items.Add(s)
```

Add, remove, or rename the options in this list. Whatever you type here appears in the dropdown.

PART 6 — The File List (ListView / Tab Area)

Search for: `def _make_listview`

6A. Column Names and Widths

python

```
cols = [  
    ("File Name", 380), # column header text, width in pixels  
    ("Status", 90),  
    ("Size (MB)", 90),  
    ("Modified", 150),  
    ("Date Added", 150),  
    ("Notes", 250),  
    ("Full Path", 400),  
]
```

To rename a column: Change the text in quotes — e.g. `"File Name"` → `"Document Name"` **To resize a column:** Change the number — e.g. `380` → `500` **Column order:** The order here is the order displayed left-to-right.

Note: If you reorder columns here, you must also update the column index numbers in `_populate_listview` where SubItems are assigned. Ask for help with that if needed.

6B. File List Font

Search for: `lv.Font = _FONT_BOLD` (inside `_make_listview`)

```
python
```

```
lv.Font = _FONT_BOLD    # currently bold - change to _FONT_NORMAL for regular weight
```

6C. Grid Lines

```
python
```

```
lv.GridLines = True    # change to False to hide the row separator lines
```

6D. Row Colors (Ready vs Missing)

Search for: `if status == "MISSING":` (inside `_populate_listview`)

```
python
```

```
if status == "MISSING":
    item.ForeColor = _CLR_MISSING    # red text for missing files
    item.Font      = _FONT_BOLD      # bold for missing files
else:
    item.ForeColor = Drawing.Color.Black    # black text for ready files
    item.Font      = _FONT_NORMAL          # normal weight for ready files
```

Change `_CLR_MISSING` to any color, or change `Drawing.Color.Black` to color-code ready files differently.

6E. Tab Labels (Section Names)

Search for: `SECTION_LABELS` in `repository_helpers.py` (not `main.py`):

```
python
```

```
SECTION_LABELS = {
    "main_wb_database":      "Main WB Database",
    "supplemental_wb_database": "Supplemental WB Database",
    "customer_provided_data": "Customer Provided Data",
}
```

Change the text on the right side of each line to rename the tabs.

6F. Tab Label with File Count Format

Search for: `def _update_tab_labels` in main.py

```
python

# With missing files:
u"{0} [{1} \u2718{2}]" # e.g. "Main WB Database [5 X2]"

# All files present:
u"{0} [{1}]"           # e.g. "Main WB Database [5]"
```

`{0}` = section name, `{1}` = total count, `{2}` = missing count. `\u2718` is the **X** symbol. Change to `\u26A0` for ⚠ or just `!"` for plain text.

PART 7 — The Status Bar (bottom of window)

Search for: `self._status_lbl.Text = "Ready"`

```
python

self._status_lbl.Text = "Ready" # default text shown when window opens
```

The status bar updates automatically during operations (add, refresh, etc.) with messages like "Loaded — 5 file(s), 0 missing". Those messages are set in `_refresh_all` and `_on_add` — search for `_set_status()` to find them all.

PART 8 — The Notes Dialog (popup when clicking Notes button)

Search for: `class NotesDialog`

```
python
```

```

self.Text    = "File Notes"      # title bar of the dialog
self.Width   = 480               # dialog width
self.Height  = 260              # dialog height

# The text box inside:
self._tb.Size = Drawing.Size(440, 140)  # width, height of the notes text area

# Buttons:
btn_ok = _make_btn("OK",        260, 185, 90, 32, primary=True, ...)
btn_cn = _make_btn("Cancel",    360, 185, 90, 32, ...)

```

PART 9 — The Health Check Dialog

Search for: `class HealthCheckDialog`

```

python

self.Text    = "Repository Health Check"  # title bar
self.Width   = 700
self.Height  = 480

# Result list columns:
lv.Columns.Add("Section",    160)
lv.Columns.Add("File Name",  200)
lv.Columns.Add("Path",       270)

# Status icons:
icon = u"\u2718 ISSUES FOUND"  # ✗ ISSUES FOUND (when problems exist)
icon = u"\u2714 ALL FILES OK"  # ✓ ALL FILES OK (when clean)

```

PART 10 — Quick Reference: Common Customizations

What you want to change	Search for this in Notepad++	What to edit
Window opens bigger	<code>self.Width = 1360</code>	Change 1360 and 900
Title bar text	<code>"Analysis Repository Manager"</code>	Change the quoted text

What you want to change	Search for this in Notepad++	What to edit
Main heading color	<code>_CLR_ANSYS_BLUE</code>	Change the RGB numbers
All fonts larger	<code>_FONT_NORMAL = Drawing.Font</code>	Change 9.5 to 11
Button colors	<code>_CLR_ANSYS_BLUE</code>	Change the RGB numbers
Add a new status option	<code>"Active", "In Review"</code>	Add to the list
Rename a tab	<code>SECTION_LABELS</code> in repository_helpers.py	Change the quoted text
Hide grid lines	<code>lv.GridLines = True</code>	Change to False
Missing file color	<code>_CLR_MISSING</code>	Change the RGB numbers
Column widths	<code>("File Name", 380)</code>	Change the number
Panel background	<code>FromArgb(245, 248, 252)</code> (in ProjectInfoPanel)	Change RGB

Tips for Editing Safely

1. **Always commit in GitHub Desktop before making changes** — type a message like "before layout edits" so you have a clean rollback point.
2. **Change one thing at a time** — save, reload extension in Workbench, test, then move on. This way if something breaks you know exactly what caused it.
3. **Use Ctrl+F in Notepad++** to find the search terms listed in each section.
4. **Reload the extension** after every save: Workbench → Extensions → Manage Extensions → Reload
5. **Check the log** at `C:\Temp\AnalysisHub_debug.log` if something doesn't look right — it will show any Python errors from your edits.